Parallax Propeller 2
Spin2 Language Documentation
2021-03-23
v35L

Document Status

Version	Date	Progress
	2020_02_06	Started document.
v34t	2020_07_15	DEBUG added, documentation up-to-date.
v34u	2020_07_19	DEBUG improved, documentation up-to-date.
v35	2020_11_18	DEBUG improved with anti-aliasing throughout, QSIN / QCOS added, documentation up-to-date.
v35e	2021_01_06	DEBUG_BAUD symbol added. Spin2 stack-locating bug fixed.
v35f	2021_01_29	DEBUG fixes. Was erring at 63 DEBUGs, now goes to 255. Was not always resetting the DEBUG.log file.
v35g	2021_02_13	DEBUG fixes. Line-clipping routine was causing floating-point exceptions and memory-access violations.
v35h	2021-02-15	• The first 16 LUT registers in the Spin2 interpreter were freed to allow for streamer 'imm-->LUT' usage. This is intended to support 1/2/4-bit video, via interrupt, within the same cog that the interpreter is running in. The inline-PASM limit went from $134 down to $124, in order to compensate. • A new DEBUG_WINDOWS_OFF symbol was added to inhibit any DEBUG windows from opening after a download. DEBUG_BAUD can now be set to alter the baud rate that DEBUG uses with PNut.exe.
v35i	2021-02-20	• Added command-line DEBUG-only mode for presenting flash-programmed DEBUG data and displays. • Fixed Floating-point error in SCOPE_XY.
v35j	2021-03-16	Fixed problem with DEBUG_BAUD <> 2_000_000 not working on some boards.
v35k	2021-03-19	Added DOWNLOAD_BAUD to existing DEBUG_BAUD for overriding default 2 Mbaud download and DEBUG.
v35L	2021-03-23	Added complete command-line interface to PNut.exe and included batch files for invoking PNut.exe and returning error status to STDOUT, STDERR, and ERRORLEVEL. See "Command Line options for PNut.exe".

OVERVIEW

The Spin2 language is designed to be very simple, but highly capable. Spin2 does not hide the underlying binary phenomena that makes computers work, but allows you to exploit it for effective programming. Assembly language is also supported in Spin2, as both in-line sequences and stand-alone programs.

A person with programming experience will be able to get a solid understanding of Spin2 in a very short amount of time. Learning Spin2 will pay dividends by allowing you to focus on your ideas, without having to navigate a myriad of typecasts and usage rules. Your brain will delight in staying busy, with compile+download+execute times of under 1 second.

In Spin2:

- There are no variable types, just three word sizes: BYTE (8 bits), WORD (16 bits), and LONG (32 bits), with bit fields supported for each.
- All math operations are performed at 32 bits and there are both signed and unsigned operators for where distinctions matter.
- Programs, called objects, can easily incorporate many other objects written by other authors with no foreknowledge of your project.
- Objects compile to compact, hardware-accelerated bytecode blocks which trigger short sequences of cog-resident interpreter code.
- Code is case-insensitive, so you can capitalize how you'd like.
- Symbolic names can be up to 32 characters in length.

In this documentation, all keywords are in UPPERCASE for clarity and anything in lowercase represents a user-defined symbolic name.

Program Structure

Spin2 programs are built from one or more objects. Objects are files which contain at least one public method, along with optional constants, child objects, variables, additional methods, and data. Objects are assembled together into a top-level object with an internal hierarchy of sub-objects. Each object instance, at run-time, gets its own set of variables, as defined by the object, to maintain its unique operating state.

Different parts of an object are declared within blocks, which all begin with 3-letter block identifiers.

The compiler can actually generate PASM-only programs, as well as Spin2+PASM programs, depending upon which blocks are present in the .spin2 file.

Block Identifier	Block Contents	Spin2+PASM Programs	PASM-only Programs
CON	Constant declarations (CON is the initial/default block type)	Permitted	Permitted
OBJ	Child-object instantiations	Permitted	Not Allowed
VAR	Variable declarations	Permitted	Not Allowed
PUB	Public method for use by the parent object and within this object	Required	Not Allowed
PRI	Private method for use within this object	Permitted	Not Allowed
DAT	Data declarations, including PASM code	Permitted	Required

Here are some minimal Spin2 and PASM-only programs. If you copy and paste these into PNut.exe, you can hit F10 to run them.

Minimal Spin2 Program

```
PUB MinimalSpin2Program()          'first PUB method executes

  REPEAT
    PINWRITE(63..56, GETRND())     'write a random pattern to P63..P56
    WAITMS(100)                    'wait 1/10th of a second, loop
```

Minimal PASM Program

```
DAT     ORG                        'start PASM at hub $00000 for cog $000

loop    DRVRND  #56 ADDPINS 7      'write a random pattern to P63..P56
        WAITX   ##clkfreq_/10      'wait 1/10th of a second, loop
        JMP     #loop
```

Here is a Spin2 program which contains every block type.

All-Block Spin2 Program

```
CON _clkfreq = 297_000_000                      'set clock frequency

OBJ vga : "VGA_640x480_text_80x40"              'instantiate vga object

VAR time, i                                     'declare object-wide variables


PUB go()                                        'this first public method executes, cog stops after

  vga.start(8)                                  'start vga on base pin 8

  SEND := @vga.print                            'establish SEND pointer

  SEND(4, $004040, 5, $00FFFF)                  'set light cyan on dark cyan

  time := GETCT()                               'capture time

  i := @text                                    'print file to vga screen
  REPEAT @textend-i
    SEND(byte[i++])

  time := GETCT() - time                        'capture time delta in clock cycles

  time := MULDIV64(time, 1_000_000, clkfreq)    'get time delta in microseconds

  SEND(12, "Time elapsed during printing was ", dec(time), " microseconds.")  'print time delta


PRI dec(value) | flag, place, digit             'private method prints decimals, three local variables

  flag~                                         'reset digit-printed flag
  place := 1_000_000_000                        'start at the one-billion's place and work downward
  REPEAT
    IF flag ||= (digit := value / place // 10) || place == 1      'print a digit?
      SEND("0" + digit)                                           'yes
      IF LOOKDOWN(place : 1_000_000_000, 1_000_000, 1_000)        'also print a comma?
        SEND(",")                                                 'yes
  WHILE place /= 10                             'next place, done?


DAT

text   FILE   "VGA_640x480_text_80x40.txt"      'include raw file data for printing
textend
```

A breakdown of each block type follows.

CON Blocks

CON blocks are used to define symbolic constants which can be used throughout the file.

- Symbolic constants resolve to 32-bit values.
- Symbolic constants can be assigned using '=' or by just expressing their names in an enumeration list.
- Symbolic constants can be referenced by every block within the file, including CON blocks.
- Symbolic constants can be referenced by the parent object's methods via 'objectname.constantname' syntax.
- If a decimal point is present, the value is encoded in IEEE-754 single-precision format.

CON

Direct Constant Assignments

```
CON  EnableFlow = 8                 'single assignments
     DisableFlow = 4
     ColorBurstFreq = 3_579_545

     PWM_base = 8
     PWM_pins = PWM_base ADDPINS 7

     x = 5, y = -5, z = 1           'comma-separated assignments

     HalfPi = 1.5707963268          'single-precision float values
     QuarPi = HalfPi / 2.0

     j = ROUND(4000.0 / QuarPi)     'float to integer
```

CON

Enumerated Constant Assignments

```
CON  #0,a,b,c,d           'a=0, b=1, c=2, d=3    (start=0, step=1)
     #1,e,f,g,h           'e=1, f=2, g=3, h=4    (start=1, step=1)

     #4[2],i,j,k,l        'i=4, j=6, k=8, l=10   (start=4, step=2)
     #-1[-1],m,n,p        'm=-1, n=-2, p=-3      (start=-1, step=-1)

     #16                  'start=16, step=1
     q                    'q=16
     r[0]                 'r=17   ([0] is a step multiplier)
     s                    's=17
     t                    't=18
     u[2]                 'u=19   ([2] is a step multiplier)
     v                    'v=21
     w                    'w=22

CON  e0,e1,e2             'e0=0, e1=1, e2=2      (start=0, step=1)
                          '..enumeration is reset at each CON
```

OBJ Blocks

OBJ blocks are used to instantiate child objects into the current (parent) object. Child objects' methods can be executed and their constants can be referenced by the parent object at run time.

- Up to 32 different child objects can be incorporated into a parent object.
- Child objects can be instantiated singularly or in arrays of up to 255.
- Up to 1024 child objects are allowed per parent object.

OBJ

Child-Object Instantiations

```
OBJ  vga       : "VGA_Driver"     'instantiate "VGA_Driver.spin2" as "vga"

     mouse     : "USB_Mouse"      'instantiate "USB_Mouse.spin2" as "mouse"

     v[16]     : "VocalSynth"     'instantiate an array of 16 objects
                                  '..v[0] through v[15]
```

From within a parent-object method, a child-object method can be called by using the syntax:

```
object_name.method_name({any_parameters})
```

From within a parent-object method, a child-object constant can be referenced by using the syntax:

```
object_name.constant_name
```

VAR Blocks

VAR blocks are used to declare symbolic variables which can be utilized by all methods within the object.

- Variables can be longs (32 bits), words (16 bits), and bytes (8 bits).
- Variables can be declared as singles or arrays.
- Variables are packed in memory in the order they are declared, beginning at a long-aligned address.
- Variables are initialized to zero at run time.
- Each object's first 15 longs of variable memory are accessed via special bytecodes for improved efficiency.
- Each instance of an object will require one long, plus its declared amount of VAR space, plus 0..3 bytes to long-align for the next object's variable space.

VAR

Variable Declarations

```
VAR  CogNum                 'The default variable size is LONG (32 bits).
     CursorMode
     PosX                   'The first 15 longs have special bytecodes for faster/smaller code.
     Posy
     SendPtr                'So, declare your most common variables first, as longs.

     BYTE StringChr         'byte variable (8 bits)
     BYTE StringBuff[64]    'byte variable array (64 bytes)
     BYTE a,b,c[1000],d     'comma-separated declarations

     WORD CurrentCycle      'word variable (16 bits)
     WORD Cycles[200]       'word variable array (200 words)
```

```
WORD e,f[5],g,h[10]         'comma-separated declarations

LONG Value                  'long variable
LONG Values[15]             'long variable array (15 longs)
LONG i[100],j,k,l           'comma-separated declarations

ALIGNW                      'word-align to hub memory, advances variable pointer as necessary

ALIGNL                      'long-align to hub memory, advances variable pointer as necessary
BYTE Bitmap[640*480]        '..useful for making long-aligned buffers for FIFO-wrapping
```

PUB and PRI Blocks

PUB and PRI blocks are used to define public and private executable Spin2 methods.

- PUB methods are available to the parent object, as well as to the object they are defined in.
- PRI methods are available only to the object they are defined in.
- The first PUB method in an object is what executes when that object is run as the top-level object.
- Methods can have from 0 to 127 input parameters, all of which are single longs.
- Methods can have from 0 to 15 output results, all of which are single longs.
- Methods can have up to 64KB of local variables, which can be bytes, words, and longs (default), in both singles and arrays.
- Local variable size overrides (BYTE/WORD) apply only to the variable being declared, not subsequent variables.
- Results are initialized to zero on method entry, while local variables are undefined.
- Parameters, then results, and then local variables are packed into stack memory in the order they are declared.
- In-line PASM code can access the first 16 longs of parameters...results...locals via registers with the same symbolic names.

PUB/PRI syntax is as follows:

```
PUB/PRI methodname({parameter{,...}}) {: result{,...}} {| {ALIGNW/ALIGNL} {BYTE/WORD/LONG} localvar{[count]}{,...}}
```

PUB / PRI Declarations (method code would go below each declaration)	Input Parameters (longs)	Output Results (longs)	Local Variables (longs, words, bytes)
`PUB go()`	0	0	0
`PUB SetupADC(pins)`	1	0	0
`PUB StartTx(pin, baud) : Okay`	2	1	0
`PRI RotateXY(X, Y, Angle) : NewX, NewY | p,q,r`	3	2	3 longs
`PRI Shuffle() | i, j`	0	0	2 longs
`PRI FFT1024(DataPtr) | a, b, x[1024], y[1024]`	1	0	1+1+1024+1024 longs
`PRI ReMix() : Length, SampleRate | WORD Buff[20000], k`	0	2	20000 words + 1 long
`PRI StrCheck(StrPtrA, StrPtrB) : Pass | i, BYTE Str[64]`	2	1	1 long + 64 bytes

DAT Blocks

DAT blocks are used to express data and PASM code.

- Data are packed in memory in the order they are declared, beginning at a long-aligned address.
- Data are expressed using the following syntax: {symbolname} `BYTE/WORD/LONG data{[count]} {,data...}`
- Symbols that precede data and PASM instructions resolve to addresses
  - In Spin2+PASM programs, hub addresses are relative to the start of the object and can be referenced as follows:
    - 'SymbolName' will return the data at the symbol, in accordance with its size (byte/word/long).
    - '@SymbolName' will return the address of the data.
    - '@@SymbolName' will convert an '@Symbol' in the data to an absolute address (see "DAT Data Pointers")
  - In PASM-only programs, hub addresses are absolute.

DAT
Symbols and Data

```
DAT                                         'symbols without data take the size of the previous declaration

HexChrs         BYTE    "0123456789ABCDEF"  'HexChrs is a byte symbol that points to "0"
symbol0                                     'symbol0 is a byte symbol that points after "F"

Pattern         WORD    $CCCC,$3333,$AAAA,$5555 'Pattern is word symbol that points to $CCCC
symbol1                                     'symbol1 is a word symbol that points after $55555

Billions        LONG    1_000_000_000       'Billions is a long symbol that points to 1_000_000_000
symbol2                                     'symbol2 is a long symbol that points after 1_000_000_000

DoNothing       NOP                         'DoNothing is a long symbol that points to a NOP instruction
symbol3                                     'symbol3 is a long symbol that points after the NOP instruction

symbol4         BYTE                        'symbol4 is a byte symbol that points to $78
symbol5         WORD                        'symbol5 is a word symbol that points to $5678
symbol6         LONG                        'symbol6 is a long symbol that points to $12345678

                LONG    $12345678           'long value $12345678

                BYTE    100[64]             '64 bytes of value 100

                BYTE    10, WORD 500, LONG $FC000 'BYTE/WORD/LONG overrides allowed for single values
                BYTE    FVAR 99, FVARS -99  'FVAR/FVARS overrides allowed, can be read via RFVAR/RFVARS

FileDat         FILE    "Filename"          'include binary file, FileDat is a byte symbol that points to file

                ALIGNW                      'word-align to hub by emitting a zero byte, if necessary
                ALIGNL                      'long-align to hub by emitting 1 to 3 zero bytes, if necessary
```

DAT
Data Pointers

```
DAT

Str0            BYTE    "Monkeys",0    'strings with symbols
Str1            BYTE    "Gorillas",0
Str2            BYTE    "Chimpanzees",0
Str3            BYTE    "Humanzees",0

StrList         WORD    @Str0          'in Spin2, these are offsets of strings relative to start of object
                WORD    @Str1          'in Spin2, @@StrList[i] will return address of Str0..Str3 for i = 0..3
                WORD    @Str2          'in PASM-only programs, these are absolute addresses of strings
                WORD    @Str3          '(use of WORD supposes offsets/addresses are under 64KB)
```

DAT
Cog-exec

```
DAT             ORG                    'begin a cog-exec program (no symbol allowed before ORG)
                                       'COGINIT(16, @IncPins, 0) will launch this program in a free cog
IncPins         MOV     DIRA,#$FF      'to Spin2 code, IncPins is the 'MOV' instruction (long)
Loop            ADD     OUTA,#1        'to Spin2 code, @IncPins is the hub address of the 'MOV' instruction
                JMP     #Loop          'to PASM code, Loop is the cog address ($001) of the 'ADD' instruction

                ORG                    'set cog-exec mode, cog address = $000, cog limit = $1F8 (reg, both defaults)
                ORG     $100           'set cog-exec mode, cog address = $100, cog limit = $1F8 (reg, default limit)
                ORG     $120,$140      'set cog-exec mode, cog address = $120, cog limit = $140 (reg)
                ORG     $200           'set cog-exec mode, cog address = $200, cog limit = $400 (LUT, default limit)
                ORG     $300,$380      'set cog-exec mode, cog address = $300, cog limit = $380 (LUT)

                ADD     reg,#1         'in cog-exec mode, instructions force alignment to cog/LUT registers

                ORGF    $040           'fill to cog address $040 with zeros (no symbol allowed before ORGF)

                FIT     $020           'test to make sure cog address has not exceeded $020

x               RES     1              'reserve 1 register, advance cog address by 1, don't advance hub address
y               RES     1              'reserve 1 register, advance cog address by 1, don't advance hub address
z               RES     1              'reserve 1 register, advance cog address by 1, don't advance hub address
buff            RES     16             'reserve 16 registers, advance cog address by 16, don't advance hub address
```

DAT
Hub-exec

```
DAT             ORGH                   'begin a hub-exec program (no symbol allowed before ORGH)
                                       'COGINIT(32+16, @IncPins, 0) will launch this program in a free cog
IncPins         MOV     DIRA,#$FF      'In Spin2, IncPins is the 'MOV' instruction (long)
Loop            ADD     OUTA,#1        'In Spin2, @IncPins is the hub address of the 'MOV' instruction
                JMP     #Loop          'In PASM, Loop is the hub address ($00404) of the 'ADD' instruction

                ORGH                   'set hub-exec mode, hub origin = $00400, origin limit = $100000 (both defaults)
                ORGH    $1000          'set hub-exec mode, hub origin = $01000, origin limit = $100000 (default limit)
                ORGH    $FC000,$FC800  'set hub-exec mode, hub origin = $FC000, origin limit = $FC800

                FIT     $2000          'test to make sure hub address has not exceeded $2000
```

There are some differences between Spin2+PASM programs and PASM-only programs, when it comes to hub-exec code:

Spin2+PASM Programs

- Hub-exec code must use relative addressing, since it is not located at its place of origin.
- The LOC instruction can be used to get addresses of data assets within relative hub-exec code.
- ORGH must specify at least $400, so that pure hub-exec code will be assembled.
- The default ORGH address of $400 is always appropriate, unless you are writing code which will be moved to its actual ORGH address at runtime, so that it can use absolute addressing.

```
DAT     ORGH            'set hub-exec mode and set origin to $400
        ORGH    $FC000  'set hub-exec mode and set origin to $FC000
```

PASM-Only Programs

- Hub-exec code may use absolute and relative addressing, since origin always matches hub address.
- ORGH fills hub memory with zeros, up to the specified address.

```
DAT     ORGH            'set hub-exec mode at current hub address
        ORGH    $400    'set hub-exec mode and fill hub memory with zeros to $400
```

Spin2 Language

Constants

Constants resolve to 32-bit values and can be expressed as follows:

Constants	Examples	Descriptions

Decimal	`1`
	`-150`
	`3_000_000`
		• Decimal values use digits '0'..'9'
		• Underscores '_' are allowed after the first digit for placeholding

Hexadecimal	`$1B`
	`$AA55`
	`$FFFF_FFFF`
		• Hex values start with '$' and use digits '0'..'9' and 'A'..'F'
		• Underscores '_' are allowed after the first digit for placeholding

Double Binary	`%%21`
	`%%01_23`
	`%%3333_2222_1111_0000`
		• Double binary values start with '%%' and use digits '0'..'3'
		• Underscores '_' are allowed after the first digit for placeholding

Binary	`%0110`
	`%1_1111_1000`
	`%0001_0010_0011_0100`
		• Binary values start with '%' and use digits '0' and '1'
		• Underscores '_' are allowed after the first digit for placeholding

Float	`-1.0`
	`1_250_000.0`
	`1e9`
	`-1.23456e-7`
		• Float values use digits '0'..'9' and have a '.' and/or 'e' in them
		• Floats are encoded in IEEE-754 single-precision 32-bit format
		• Underscores '_' are allowed after the first digit for placeholding
		• Floats are not part of Spin2, but a library can offer floating-point functions

Character	`"H"`	• A single character in quotes resolves to a 7-bit ASCII value

Variables

In Spin2, there are both user-defined and permanent variables. The user-defined variable sources are listed below and the permanent variables are shown in the table.

- VAR variables (hub)
- PUB/PRI parameters, return values, and local variables (hub)
- DAT symbols (hub)
- Cog registers

Variables (all LONG)	Variable Name	Address or Offset	Description	Useful in Spin2	Useful in Spin2-PASM	Useful in PASM-Only
Hub Locations	`CLKMODE` `CLKFREQ`	`$00040` `$00044`	Clock mode value Clock frequency value	Yes Yes	Yes Yes	No No
Hub VAR	`VARBASE`	+0	Object base pointer, @VARBASE is VAR base, used by method-pointer calls	Maybe	No	No

Cog Registers	`PR0`
	`PR1`
	`PR2`
	`PR3`
	`PR4`
	`PR5`
	`PR6`
	`PR7`
		`$1D8`
		`$1D9`
		`$1DA`
		`$1DB`
		`$1DC`
		`$1DD`
		`$1DE`
		`$1DF`
			Spin2 <-> PASM communication	Yes
				Yes
				Yes
				Yes
				Yes
				Yes
				Yes
				Yes
					Yes
					Yes
					Yes
					Yes
					Yes
					Yes
					Yes
					Yes
						No
						No
						No
						No
						No
						No
						No
						No

	`IJMP3`
	`IRET3`
	`IJMP2`
	`IRET2`
	`IJMP1`
	`IRET1`
		`$1F0`
		`$1F1`
		`$1F2`
		`$1F3`
		`$1F4`
		`$1F5`
			Interrupt JMP's and RET's	No
				No
				No
				No
				No
				No
					Yes
					Yes
					Yes
					Yes
					Yes
					Yes
						Yes
						Yes
						Yes
						Yes
						Yes
						Yes

	`PA`
	`PB`
	`PTRA`
	`PTRB`
		`$1F6`
		`$1F7`
		`$1F8`
		`$1F9`
			Pointer registers
			
			Data pointer passed from COGINIT
			Code pointer passed from COGINIT
				No
				No
				No
				No
					Yes
					Yes
					Yes
					Yes
						Yes
						Yes
						Yes
						Yes

	`DIRA`
	`DIRB`
	`OUTA`
	`OUTB`
	`INA`
	`INB`
		`$1FA`
		`$1FB`
		`$1FC`
		`$1FD`
		`$1FE`
		`$1FF`
			Output enables for P31..P0
			Output enables for P63..P32
			Output states for P31..P0
			Output states for P63..P32
			Input states from P31..P0
			Input states from P63..P32
				Yes
				Yes
				Yes
				Yes
				Yes
				Yes
					Yes
					Yes
					Yes
					Yes
					Yes
					Yes
						Yes
						Yes
						Yes
						Yes
						Yes
						Yes

In Spin2, all variables can be indexed and accessed as bitfields. Additionally, symbolic hub variables can have BYTE/WORD/LONG size overrides:

Variable Usage	Example	Description

Plain	`AnyVar`
	`HubVar.WORD`
	`BYTE[address]`
	`REG[register]`
		Hub or permanent register variable
		Hub variable with BYTE/WORD/LONG size override
		Hub BYTE/WORD/LONG by address
		Register, 'register' may be symbol declared in ORG section

With Index	`AnyVar[index]`
	`HubVar.BYTE[index]`
	`LONG[address][index]`
	`REG[register][index]`
		Hub or permanent register variable with index
		Hub variable with size override and index
		Hub BYTE/WORD/LONG by address with index
		Register with index

With Bitfield	`AnyVar.[bitfield]`
	`HubVar.LONG.[bitfield]`
	`WORD[address].[bitfield]`
	`REG[register].[bitfield]`
		Hub or permanent register variable with bitfield
		Hub variable with size override and bitfield
		Hub BYTE/WORD/LONG by address with bitfield
		Register with bitfield

With Index and Bitfield	`AnyVar[index].[bitfield]`
	`HubVar.BYTE[index].[bitfield]`
	`LONG[address][index].[bitfield]`
	`REG[register][index].[bitfield]`
		Hub or permanent register variable with index and bitfield
		Hub variable with size override, index, and bitfield
		Hub BYTE/WORD/LONG by address with index and bitfield
		Register with index and bitfield

A bitfield is a 10-bit value which contains a base-bit number in bits 4..0 and an additional-bits number in bits 9..5. Bitfields can be defined in a few different ways:

Bitfield	Bit Range	Details
`.[%00000_00000]`	`0`	0 additional bits above the base bit 0, a single-bit bitfield
`.[%00000_11111]`	`31`	0 additional bits above the base bit 31, a single-bit bitfield

`.[%00010_01111]`	`17..15`	2 additional bits above the base bit 15, a three-bit bitfield
`.[%11110_00000]`	`30..0`	30 additional bits above the base bit 0, a 31-bit bitfield
`.[%11111_10000]`	`15..0, 31..16`	31 additional bits above the base bit 16, wraps around, a 32-bit bitfield
`.[%00001_11111]`	`0, 31`	1 additional bit above the base bit 31, wraps around, a 2-bit bitfield
`.[23]`	`23`	Just the base bit, adds no extra bits
`.[31..20]`	`31..20`	'Top..Bottom' syntax allowed within '`.[]`', wraps if Top < Bottom
`.[5 ADDBITS 7]`	`12..5`	ADDBITS can be used to compute the bitfield
`.[BitfieldCon]`	`13..9`	`CON BitfieldCon = 9 ADDBITS 4  'BitfieldCon useful in PASM, too`
`.[BitfieldVar]`	`?`	`BitfieldVar := BaseBit ADDBITS ExtraBits  'wraps if BaseBit + ExtraBits > 31`

In addition to bitfields, there are also pinfields, which are used to select a range of I/O pins within the same 32-pin block (P63..P32 or P31..P0). Pinfields are 11-bit values which contain a base-pin number in bits 5..0 and an additional-pins number in bits 10..6. Pinfields are used by instructions which interface to pins.

Pinfield	Pin Range	Details
`PINLOW(%00000_000000)`	`0`	0 additional pins above the base pin 0, a single-pin pinfield
`PINLOW(%00000_111111)`	`63`	0 additional pins above the base pin 63, a single-pin pinfield
`PINLOW(%00011_100000)`	`35..32`	3 additional pins above the base pin 32, a four-pin pinfield
`PINLOW(%11111_001000)`	`7..0, 31..8`	31 additional pins above the base pin 8, wraps around, a 32-pin pinfield
`PINLOW(19)`	`19`	Just the base pin, adds no extra pins
`PINLOW(49..40)`	`49..40`	'Top..Bottom' syntax allowed within '`.[]`', wraps if Top < Bottom
`PINLOW(11 ADDPINS 4)`	`15..11`	ADDPINS can be used to compute the pinfield
`PINLOW(PinfieldCon)`	`53..50`	`CON PinfieldCon = 50 ADDPINS 3  'PinfieldCon useful in PASM, too`
`PINLOW(PinfieldVar)`	`?`	`PinfieldVar := BasePin ADDPINS ExtraPins  'wraps if BasePin + ExtraPins > 31`

Expressions

- Run-time expressions can incorporate constants, variables, and methods' return values
- Compile-time expressions can use only constants.
- All expressions can use operators.

Here are some examples of expressions:

Expression	Details
`BYTE[i++]`	Byte pointed to by 'i', post-increment 'i'
`(digit := value / place // 10) OR place == 1`	Boolean with buried 'digit' assignment
`place /= 10`	Divide 'place' by 10
`"0" + digit`	Get 'digit' character
`PINREAD(17..12)`	Read pins 17..12

Operators

Below is a table of all the operators available for use in Spin2 methods. Compile-time expressions can use the unary, binary, ternary and float operators.

Var-Prefix Operators	Term (method only)	Priority (term)	Assign (method only)	Priority (assign)	Description	Float Exp
`++ (pre)`	`++var`	1	`++var`	1	Pre-increment	
`-- (pre)`	`--var`	1	`--var`	1	Pre-decrement	
`?? (pre)`	`??var`	1	`??var`	1	Iterate long per XORO32, return pseudo-random	
Var-Postfix Operators	**Term (method only)**	**Priority (term)**	**Assign (method only)**	**Priority (assign)**	**Description**	**Float Exp**
`(post) ++`	`var++`	1	`var++`	1	Post-increment	
`(post) --`	`var--`	1	`var--`	1	Post-decrement	
`(post) !!`	`var!!`	1	`var!!`	1	Post-logical NOT (0 → -1, non-0 → 0)	
`(post) !`	`var!`	1	`var!`	1	Post-bitwise NOT	
`(post) \`	`var\x`	1	`var\x`	1	Post-assign x	
`(post) ~`	`var~`	1	`var~`	1	Post-clear all bits	
`(post) ~~`	`var~~`	1	`var~~`	1	Post-set all bits	
Address Operators	**Term (method only)**	**Priority (term)**			**Description**	**Float Exp**
`@`	`@symbol`	1			Hub address of VAR/PUB/PRI variable or DAT symbol	
`@`	`@method`	1			Pointer to method, may be @object{[i]}.method	
`@@`	`@@x`	1			Hub address of object + x, 'DAT x long @dat_symbol'	
`#`	`#reg_symbol`	1			Register address of cog/LUT DAT symbol	
Unary Operators	**Term**	**Priority (term)**	**Assign (method only)**	**Priority (assign)**	**Description**	**Float Exp**
`!!, NOT`	`!!x`	12	`!!= var`	1	Logical NOT (0 → -1, non-0 → 0)	
`!`	`!x`	2	`!= var`	1	Bitwise NOT (1's complement)	
`-`	`-x`	2	`-= var`	1	Negate (2's complement)	✔
`ABS`	`ABS x`	2	`ABS= var`	1	Absolute value	✔
`ENCOD`	`ENCOD x`	2	`ENCOD= var`	1	Encode MSB, 0..31	
`DECOD`	`DECOD x`	2	`DECOD= var`	1	Decode, 1 << (x & $1F)	
`BMASK`	`BMASK x`	2	`BMASK= var`	1	Bitmask, (2 << (x & $1F)) - 1	
`ONES`	`ONES x`	2	`ONES= var`	1	Sum all '1' bits, 0..32	
`SQRT`	`SQRT x`	2	`SQRT= var`	1	Square root of unsigned value	
`QLOG`	`QLOG x`	2	`QLOG= var`	1	Unsigned value to logarithm {5'whole, 27'fraction}	
`QEXP`	`QEXP x`	2	`QEXP= var`	1	Logarithm to unsigned value	
Binary Operators	**Term**	**Priority (term)**	**Assign (method only)**	**Priority (assign)**	**Description**	**Float Exp**
`>>`	`x >> y`	3	`var >>= y`	17	Shift x right by y bits, insert 0's	
`<<`	`x << y`	3	`var <<= y`	17	Shift x left by y bits, insert 0's	
`SAR`	`x SAR y`	3	`var SAR= y`	17	Shift x right by y bits, insert MSB's	
`ROR`	`x ROR y`	3	`var ROR= y`	17	Rotate x right by y bits	
`ROL`	`x ROL y`	3	`var ROL= y`	17	Rotate x left by y bits	
`REV`	`x REV y`	3	`var REV= y`	17	Reverse y LSBs of x and zero-extend	
`ZEROX`	`x ZEROX y`	3	`var ZEROX= y`	17	Zero-extend above bit y	
`SIGNX`	`x SIGNX y`	3	`var SIGNX= y`	17	Sign-extend from bit y	
`&`	`x & y`	4	`var &= y`	17	Bitwise AND	
`^`	`x ^ y`	5	`var ^= y`	17	Bitwise XOR	
`|`	`x | y`	6	`var |= y`	17	Bitwise OR	
`*`	`x * y`	7	`var *= y`	17	Signed multiply	✔
`/`	`x / y`	7	`var /= y`	17	Signed divide, return quotient	✔
`+/`	`x +/ y`	7	`var +/= y`	17	Unsigned divide, return quotient	
`//`	`x // 7`	7	`var //= y`	17	Signed divide, return remainder	
`+//`	`x +// y`	7	`var +//= y`	17	Unsigned divide, return remainder	
`SCA`	`x SCA y`	7	`var SCA= y`	17	Unsigned scale, (x * y) >> 32	
`SCAS`	`x SCAS y`	7	`var SCAS= y`	17	Signed scale, (x * y) >> 30	
`FRAC`	`x FRAC y`	7	`var FRAC= y`	17	Unsigned fraction, (x << 32) / y	
`+`	`x + y`	8	`VAR += y`	17	Add	✔
`-`	`x - y`	8	`var -= y`	17	Subtract	✔
`#>`	`x #> y`	9	`var #>= y`	17	Force x => y, signed	✔
`<#`	`x <# y`	9	`var <#= y`	17	Force x <= y, signed	✔
`ADDBITS`	`x ADDBITS y`	10	`var ADDBITS= y`	17	Make bitfield, (x & $1F) | (y & $1F) << 5	
`ADDPINS`	`x ADDPINS y`	10	`var ADDPINS= y`	17	Make pinfield, (x & $3F) | (y & $1F) << 6	
`<`	`x < y`	11			Signed less than (returns 0 or -1)	✔
`+<`	`x +< y`	11			Unsigned less than (returns 0 or -1)	
`<=`	`x <= y`	11			Signed less than or equal (returns 0 or -1)	✔
`+<=`	`x +<= y`	11			Unsigned less than or equal (returns 0 or -1)	

==	x == y	11			Equal (returns 0 or -1)	✔
<>	x <> y	11			Not equal (returns 0 or -1)	✔
>=	x >= y	11			Signed greater than or equal (returns 0 or -1)	✔
+>=	x +>= y	11			Unsigned greater than or equal (returns 0 or -1)	
>	x > y	11			Signed greater than (returns 0 or -1)	✔
+>	x +> y	11			Unsigned greater than (returns 0 or -1)	
<=>	x <=> y	11			Signed comparison (<,=,> returns -1,0,1)	✔
&&, AND	x && y	13	var &&= y	17	Logical AND (x <> 0 AND y <> 0, returns 0 or -1)	
^^, XOR	x ^^ y	14	var ^^= y	17	Logical XOR (x <> 0 XOR y <> 0, returns 0 or -1)	
||, OR	x || y	15	var ||= y	17	Logical OR (x <> 0 OR y <> 0, returns 0 or -1)	
Ternary Operator	Term	Priority (term)			Description	Float Exp
? :	x ? y : z	16			If x <> 0 then choose y, else choose z	
Assign Operator			Assign (method only)	Priority (assign)	Description	Float Exp
:=			var := x v1,v2 := x,y	17	Set var to x Set v1 to x, set v2 to y, etc. ('_' on left = ignore)	
Equate Operator			Assign (CON block only)	Priority (equate)	Description	Float Exp
=			symbol = x	17	Set symbol to x in CON block	
Float Operators	Term (constant only)				Description	Float Exp
FLOAT()	FLOAT(x)				Convert integer x to float	✔
ROUND()	ROUND(x)				Convert float x to rounded integer	✔
TRUNC()	TRUNC(x)				Convert float x to truncated integer	✔

Built-in Methods

Hub Methods	Details
`HUBSET(Value)`	Execute HUBSET instruction using Value
`CLKSET(NewCLKMODE, NewCLKFREQ)`	Safely establish new clock settings, updates CLKMODE and CLKFREQ
`COGSPIN(CogNum, Method({Pars}), StkAddr)`	Start Spin2 method in a cog, returns cog's ID if used as an expression element, -1 = no cog free
`COGINIT(CogNum, PASMaddr, PTRAvalue)`	Start PASM code in a cog, returns cog's ID if used as an expression element, -1 = no cog free
`COGSTOP(CogNum)`	Stop cog CogNum
`COGID() : CogNum`	Get this cog's ID
`COGCHK(CogNum) : Running`	Check if cog CogNum is running, returns -1 if running or 0 if not
`LOCKNEW() : LockNum`	Check out a new LOCK from inventory, LockNum = 0..15 if successful or < 0 if no LOCK available
`LOCKRET(LockNum)`	Return a certain LOCK to inventory
`LOCKTRY(LockNum) : LockState`	Try to capture a certain LOCK, LockState = -1 if successful or 0 if another cog has captured the LOCK
`LOCKREL(LockNum)`	Release a certain LOCK
`LOCKCHK(LockNum) : LockState`	Check a certain LOCK's state, LockState[31] = captured, LockState[3:0] = current or last owner cog
`COGATN(CogMask)`	Strobe ATN input(s) of cog(s) according to 16-bit CogMask
`POLLATN() : AtnFlag`	Check if this cog has received an ATN strobe, AtnFlag = -1 if ATN strobed or 0 if not strobed
`WAITATN()`	Wait for this cog to receive an ATN strobe

Pin Methods	Details
`PINW | PINWRITE(PinField, Data)`	Drive PinField pin(s) with Data
`PINL | PINLOW(PinField)`	Drive PinField pin(s) low
`PINH | PINHIGH(PinField)`	Drive PinField pin(s) high
`PINT | PINTOGGLE(PinField)`	Drive and toggle PinField pin(s)
`PINF | PINFLOAT(PinField)`	Float PinField pin(s)
`PINR | PINREAD(PinField) : PinStates`	Read PinField pin(s)
`PINSTART(PinField, Mode, Xval, Yval)`	Start PinField smart pin(s): DIR=0, then WRPIN=Mode, WXPIN=Xval, WYPIN=Yval, then DIR=1
`PINCLEAR(PinField)`	Clear PinField smart pin(s): DIR=0, then WRPIN=0
`WRPIN(PinField, Data)`	Write 'mode' register(s) of PinField smart pin(s) with Data
`WXPIN(PinField, Data)`	Write 'X' register(s) of PinField smart pin(s) with Data
`WYPIN(PinField, Data)`	Write 'Y' register(s) of PinField smart pin(s) with Data
`AKPIN(PinField)`	Acknowledge PinField smart pin(s)
`RDPIN(Pin) : Zval`	Read Pin smart pin and acknowledge, Zval[31] = C flag from RDPIN, other bits are RDPIN data
`RQPIN(Pin) : Zval`	Read Pin smart pin without acknowledge, Zval[31] = C flag from RQPIN, other bits are RQPIN data

Timing Methods	Details
`GETCT() : Count`	Get 32-bit system counter
`POLLCT(Tick) : Past`	Check if system counter has gone past 'Tick', returns -1 if past or 0 if not past
`WAITCT(Tick)`	Wait for system counter to get past 'Tick'
`WAITUS(Microseconds)`	Wait Microseconds, uses CLKFREQ
`WAITMS(Milliseconds)`	Wait Milliseconds, uses CLKFREQ
`GETSEC() : Seconds`	Get seconds since booting, uses 64-bit system counter and CLKFREQ, rolls over every 136 years.
`GETMS() : Milliseconds`	Get milliseconds since booting, uses 64-bit system counter and CLKFREQ, rolls over every 49.7 days.

PASM interfacing	Details
`CALL(RegOrHubAddr)`	CALL PASM code at Addr, PASM code should avoid registers $130..$1D7 and LUT
`REGEXEC(HubAddr)`	Load a self-defined chunk of PASM code at HubAddr into registers and CALL it. See REGEXEC description.
`REGLOAD(HubAddr)`	Load a self-defined chunk of PASM code or data at HubAddr into registers. See REGLOAD description.

Math Methods	Details
`ROTXY(x, y, angle32bit) : rotx, roty`	Rotate (x,y) by angle32bit and return rotated (x,y)
`POLXY(length, angle32bit) : x, y`	Convert (length,angle32bit) to (x,y)